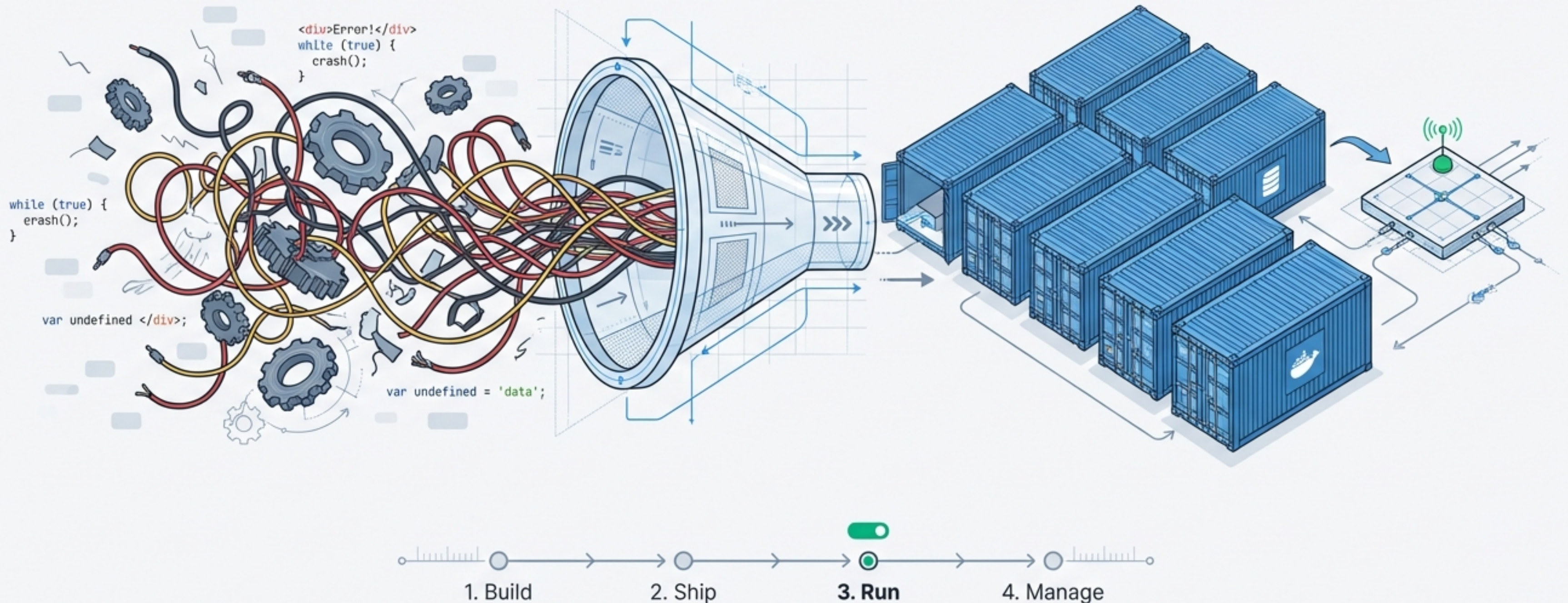


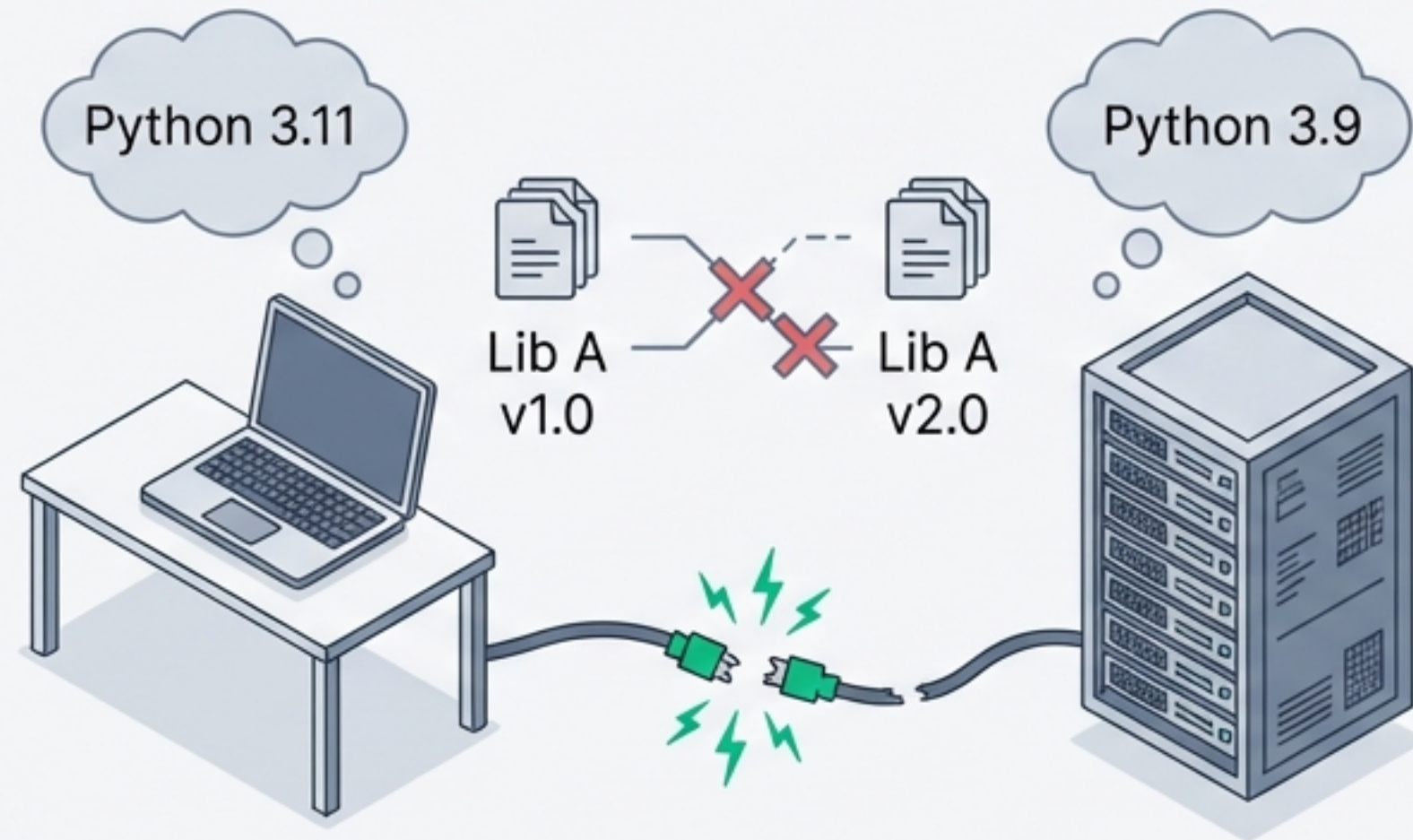
Maîtriser Docker : De l'Architecture au Déploiement

Le guide complet pour transformer le code en services résilients et portables.



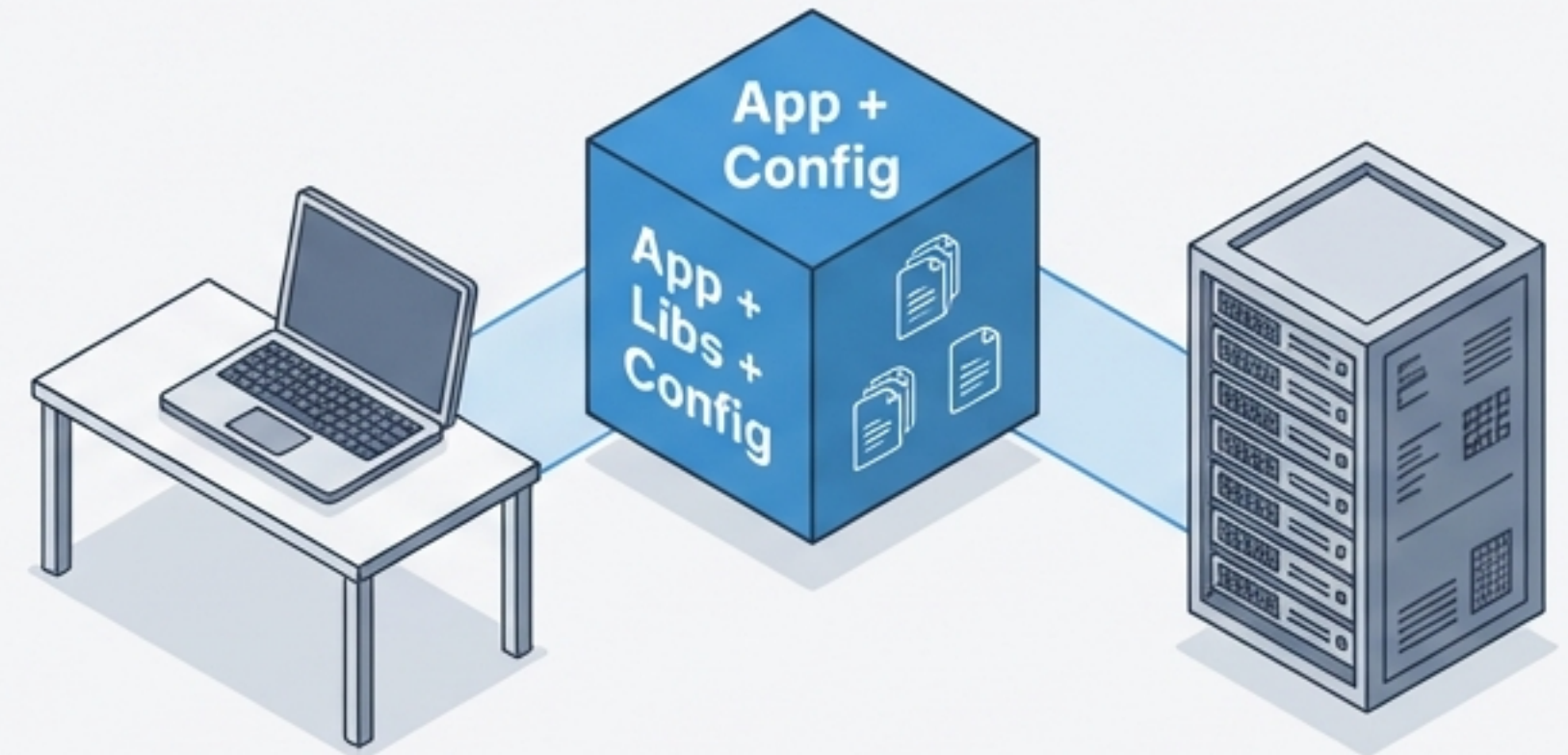
La fin du syndrome "Ça marche sur ma machine"

Le Scénario Classique (L'Enfer)



Conflits de dépendances. Environnements incohérents. Le déploiement est un pari risqué.

Le Scénario Conteneur (La Standardisation)

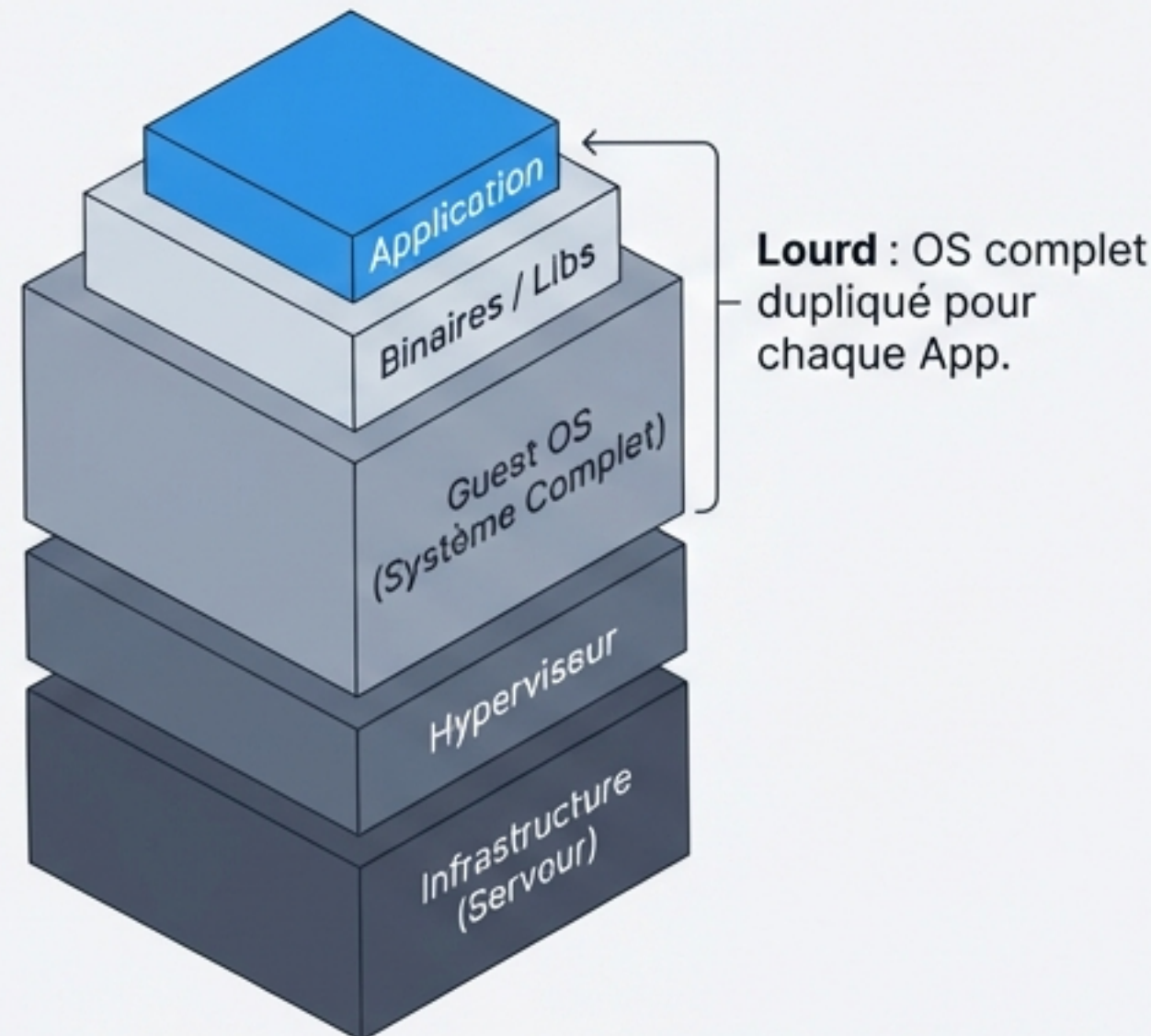


Tout est inclus. Isolation totale. L'exécution est identique à Shanghai, Rotterdam ou sur AWS.

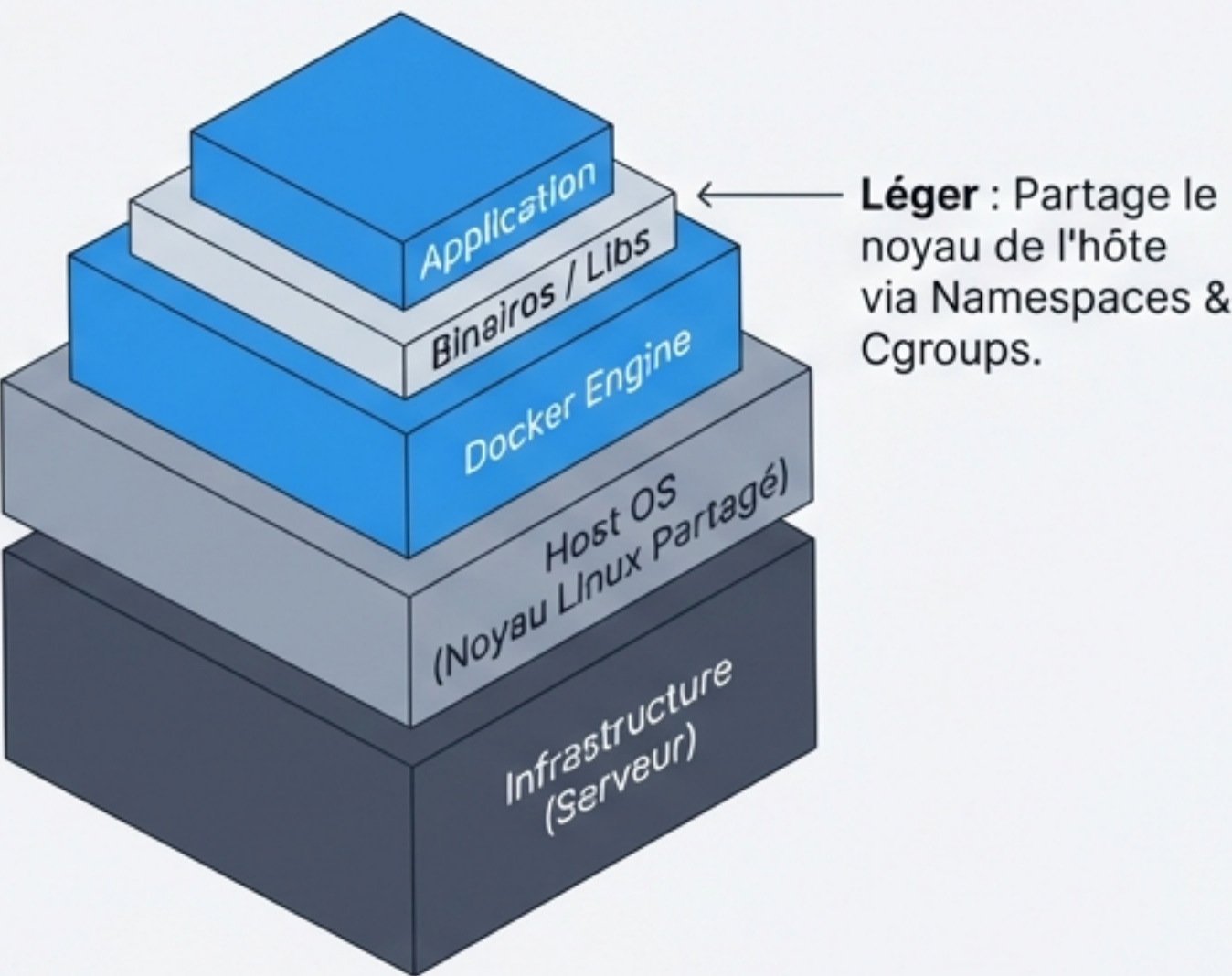
Analogie : Docker est au code ce que le conteneur maritime est au transport mondial. On ne manipule plus des objets en vrac, mais une boîte standardisée.

Isolation logicielle vs Isolation matérielle

Machine Virtuelle (VM)

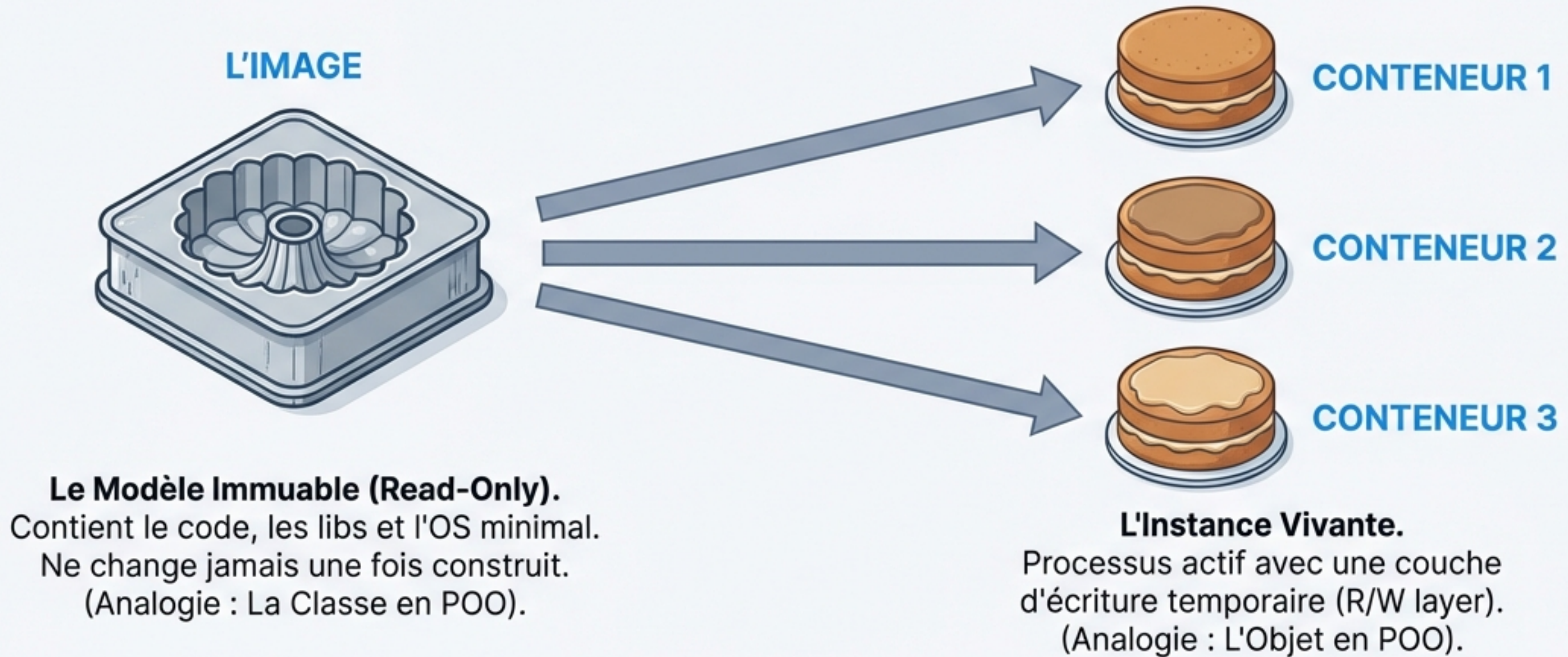


Conteneur Docker



Poids	Gigaoctets (Maison complète)	Mégaoctets (Meubles uniquement)
Démarrage	Minutes (Boot OS)	Secondes (Processus isolé)
Densité	10-20 par serveur	Des centaines par serveur

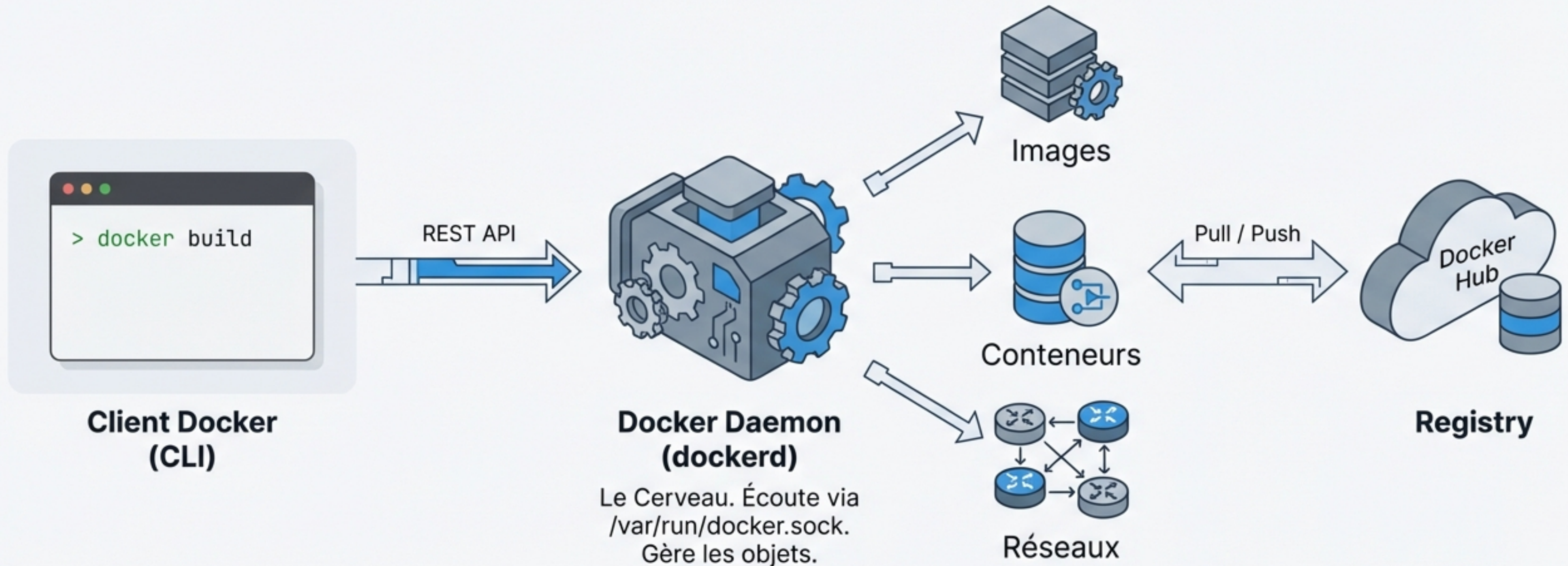
Le Modèle et l'Instance : Image vs Conteneur



```
docker run = docker create (création du layer) + docker start (lancement du PID 1)
```

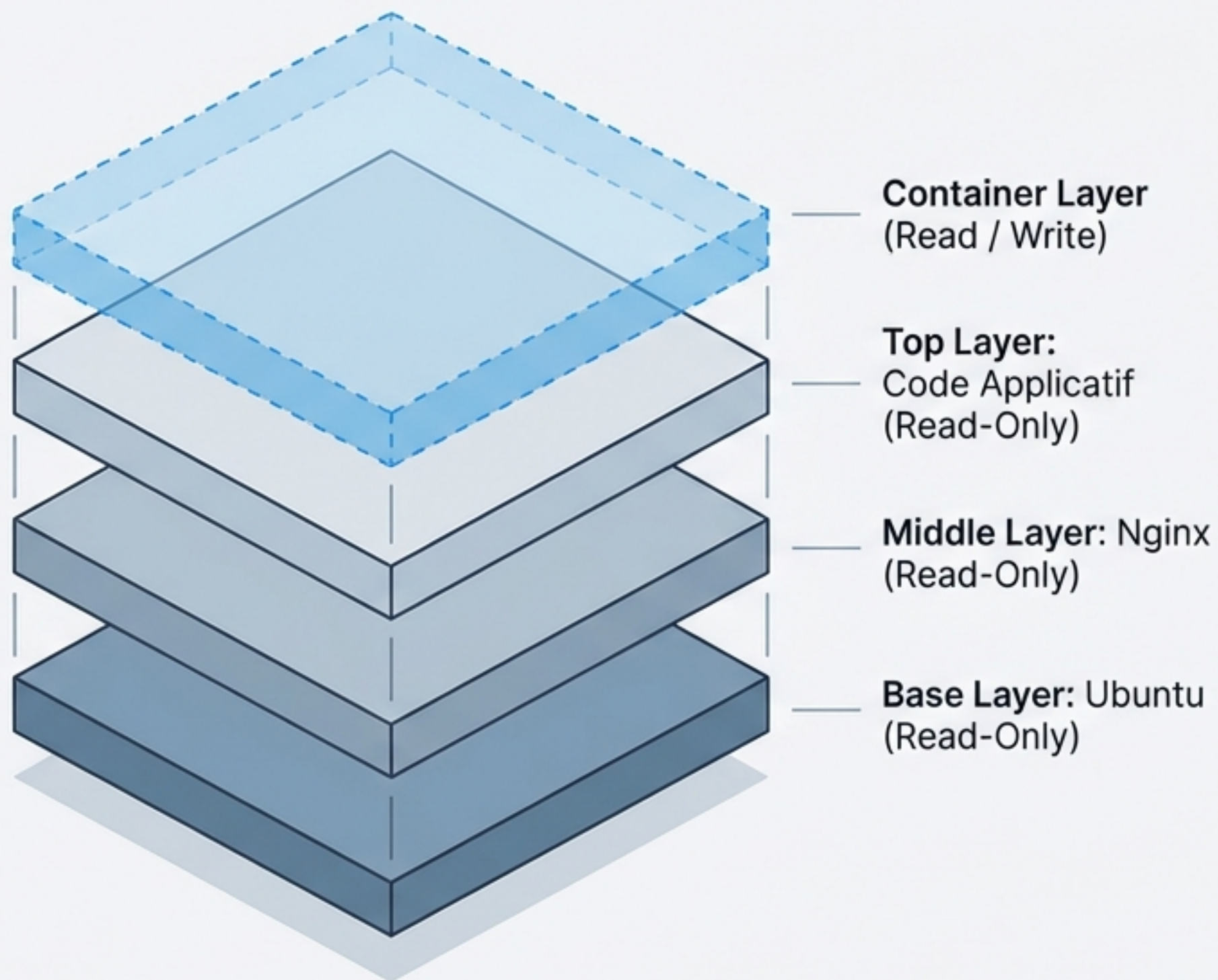
Note Clé : Règle d'or : Une image est unique, mais elle peut générer une infinité de conteneurs.

Sous le capot : L'Architecture Client-Serveur



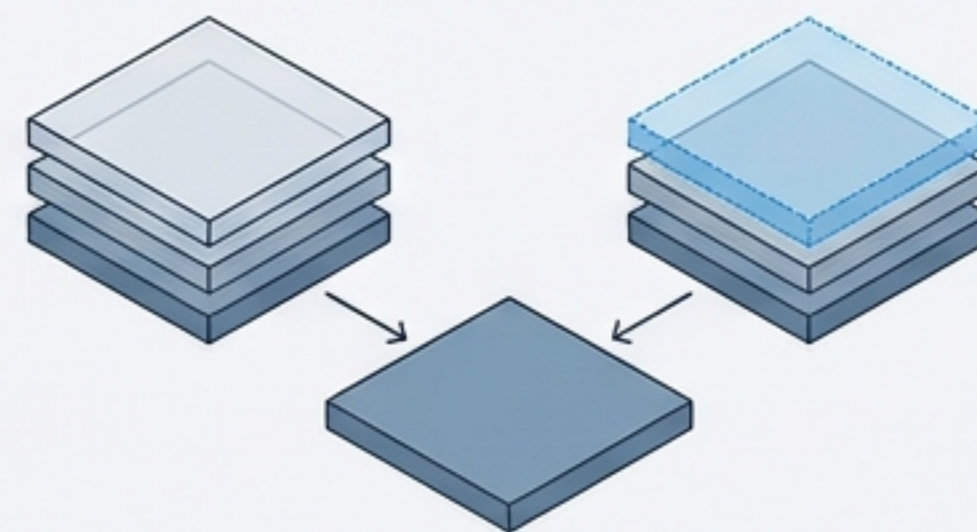
Sécurité : Le Socket Docker donne un accès équivalent à root. N'ajoutez au groupe 'docker' que des utilisateurs de confiance.

La magie des Layers (Couches)



Pourquoi cette architecture ?

1. **Cache & Vitesse** : Docker ne télécharge que les couches manquantes (delta).
2. **Efficacité de Stockage** : Plusieurs images partagent les mêmes couches de base.
3. **Minus-on-Write (CoW)** : Plusieurs images partagent les mêmes couches de base.
3. **Copy-on-Write (CoW)** : Si un fichier d'une couche basse doit être modifié, il est d'abord copié dans la couche haute.



Partage de couches & gain de place

L'Atelier de construction : Dockerfile & Build

```
FROM node:20-alpine (Annotation: Image de base légère)

WORKDIR /app

COPY . . (Annotation: Injection du code)

RUN npm install (Annotation: Installation dépendances)

CMD ['node', 'app.js'] (Annotation: Démarrage)
```

Dockerfile

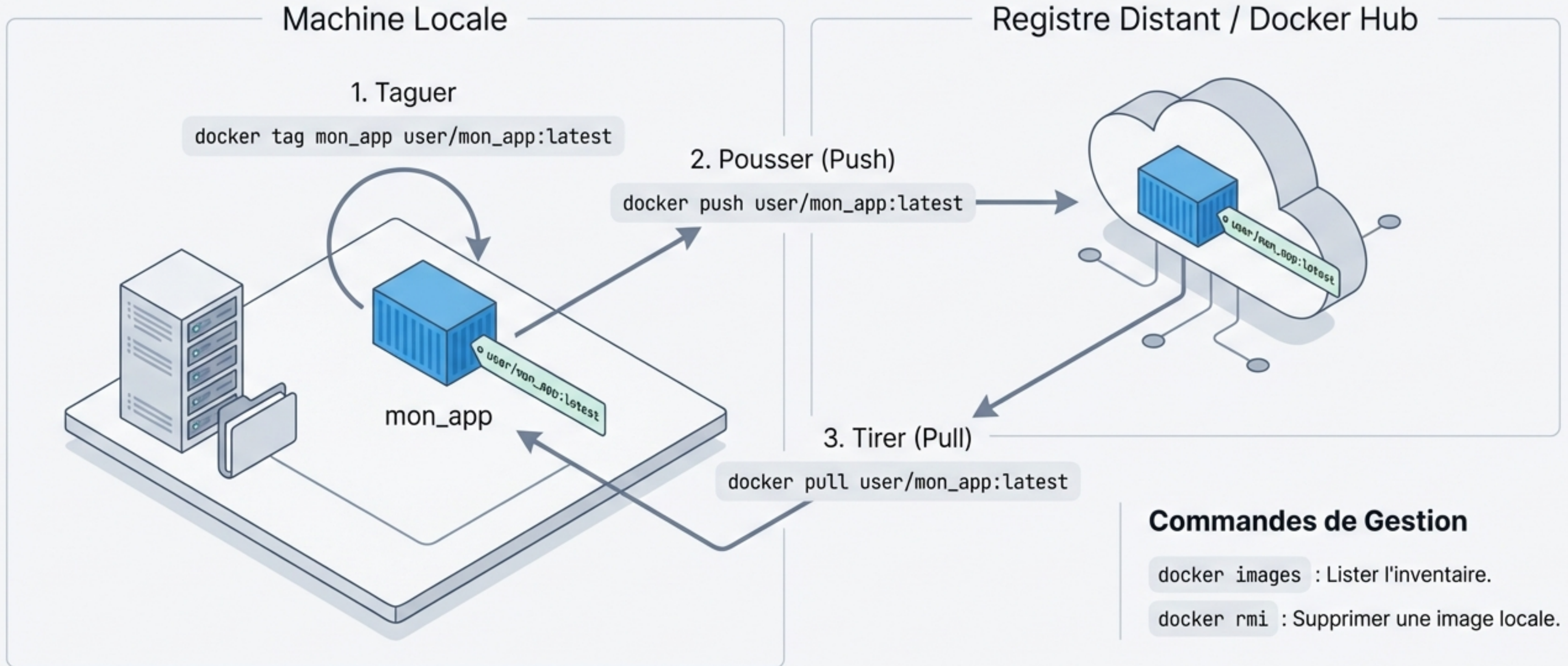
`docker build -t mon_app .`



Image: mon_app

Astuce Pro : Utilisez le Multi-stage build pour ne garder que l'essentiel en production (compiler dans une image, exécuter dans une autre).

La logistique : Taguer et Publier

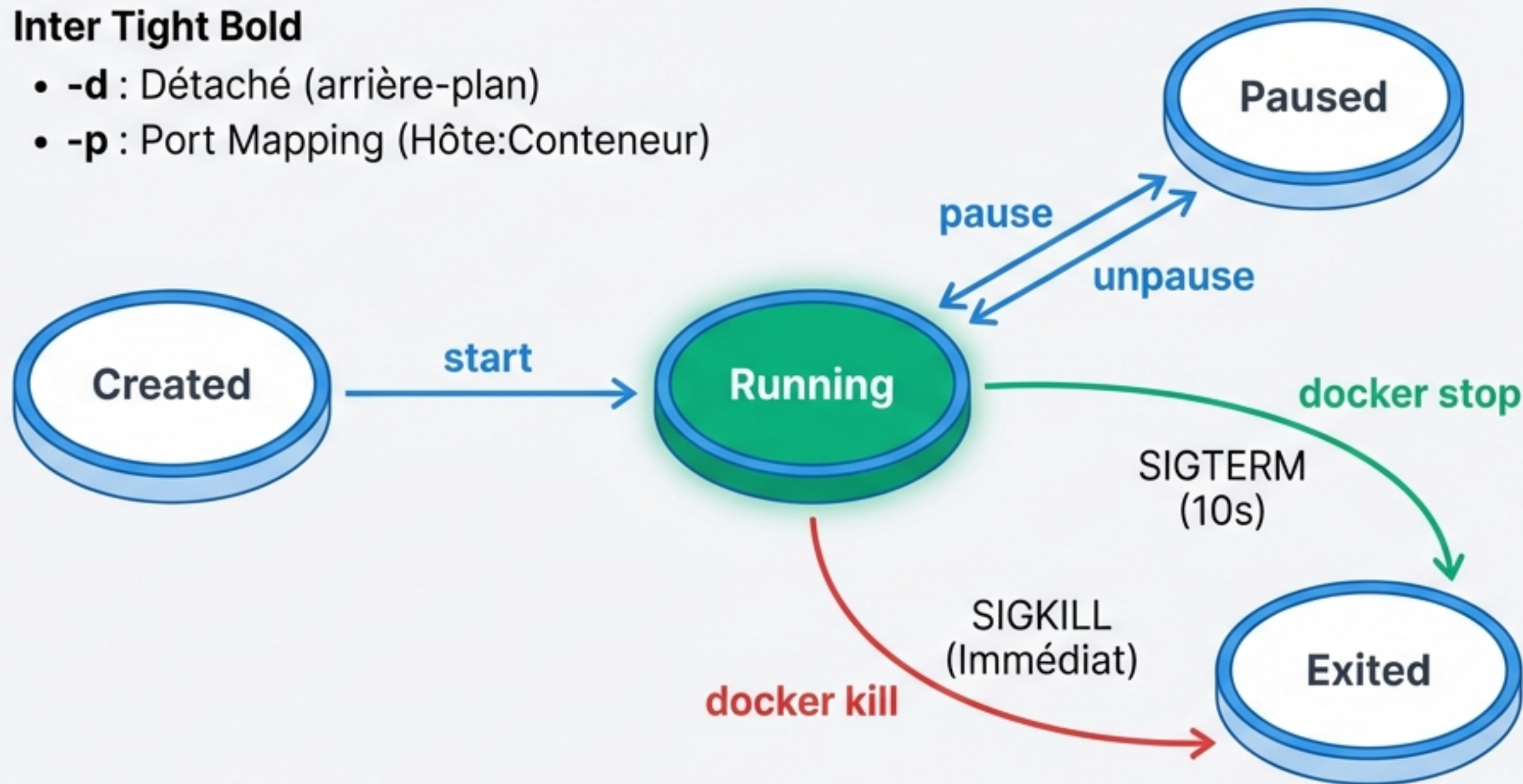


Cycle de Vie : Run, Stop, Kill

```
docker run -d --name web -p 80:80 nginx
```

Inter Tight Bold

- **-d** : Détaché (arrière-plan)
- **-p** : Port Mapping (Hôte:Conteneur)



Inter Tight Bold

Running : Le processus (PID 1) est actif.

Exited : Le processus est mort ou fini.

`docker ps` : Vérifier l'état.

L'Investigation : Debug et Monitoring

Logs (La Boîte Noire)



```
docker logs -f [id]
```

Voir stdout/stderr en temps réel.
Crucial pour comprendre les crashes.

Accès (L'Intrusion)



```
docker exec -it [id] /bin/bash
```

Entrer dans le conteneur pour
explorer le système de fichiers.

Métadonnées (L'Inspecteur)



```
docker inspect [id]
```

Tout savoir sur l'IP, les volumes,
et l'environnement.

Santé (Le Vital)



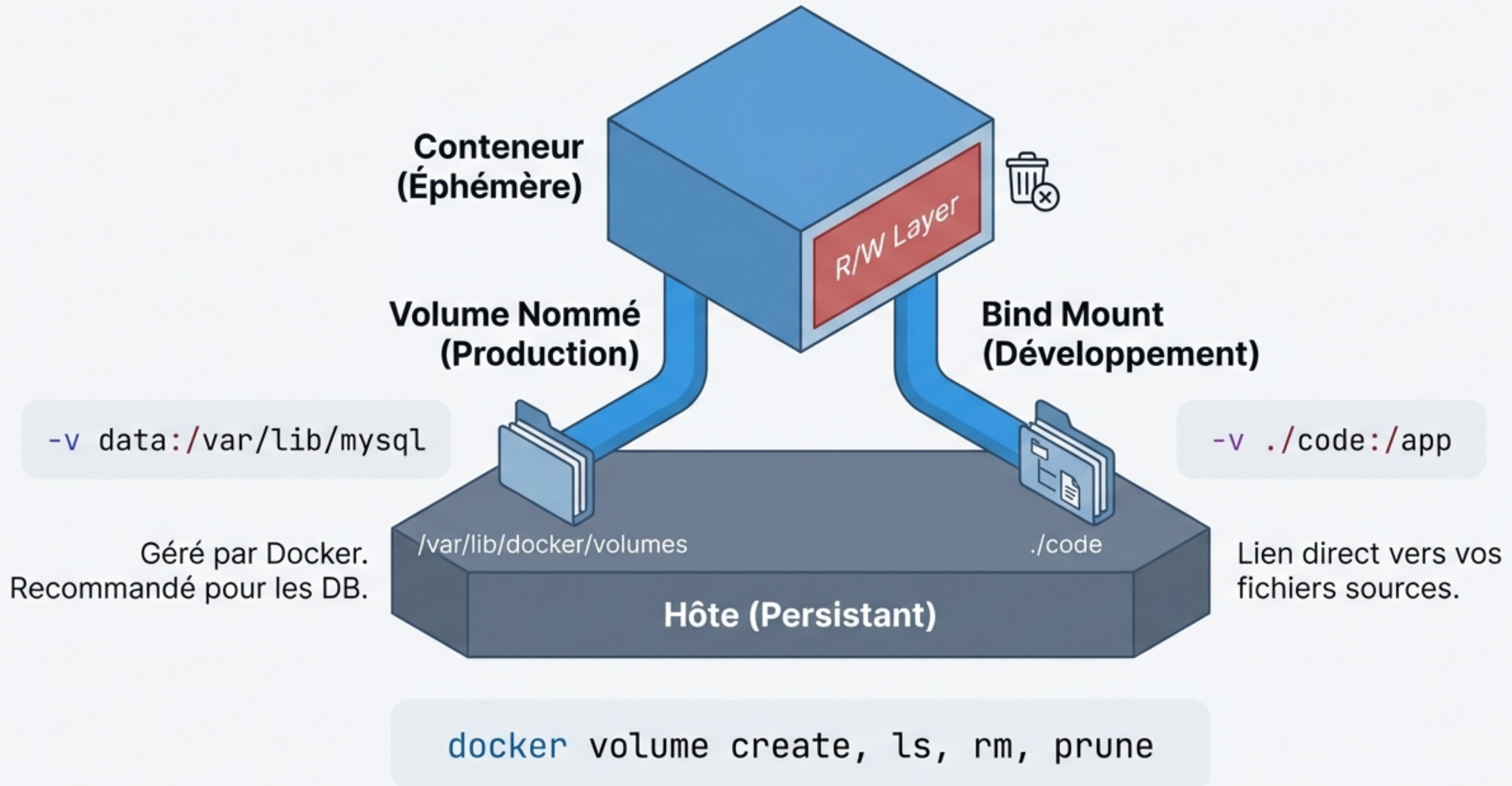
```
docker stats
```

Monitoring live (CPU / RAM).

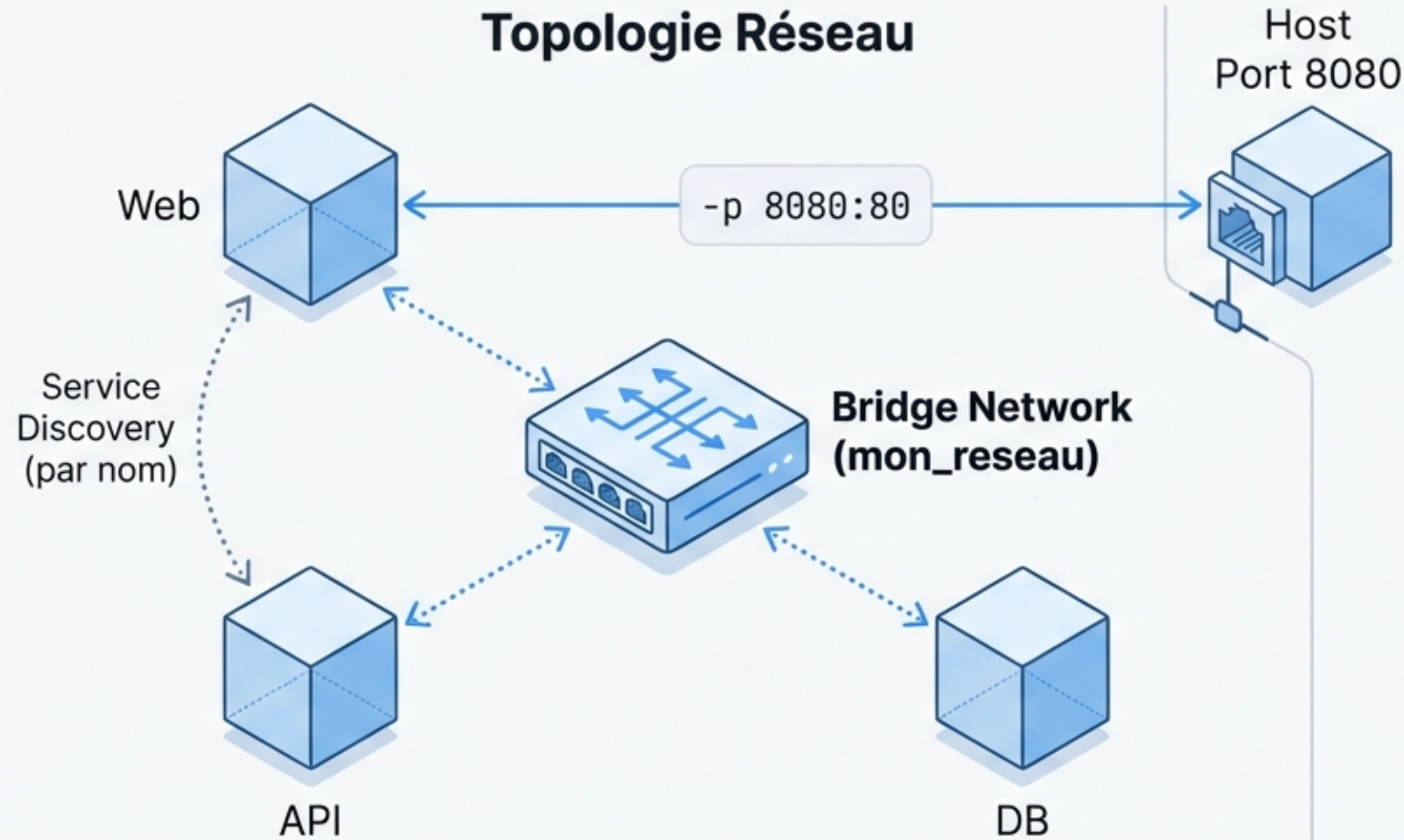
Troubleshooting

Erreur courante : Code de sortie 137 = SIGKILL. Souvent causé par l'OOM Killer (Manque de RAM).

La Mémoire : Volumes et Persistance



La Communication : Réseaux Docker



Modes Réseau

1. **Bridge (Défaut)** : Réseau privé virtuel.
2. **Host** : Partage l'IP de l'hôte (Perf max, isolation nulle).
3. **None** : Isolation totale.

Commandes

```
docker network create  
<nom>  
  
docker network connect  
<réseau> <conteneur>
```


L'Orchestration Simplifiée : Docker Compose

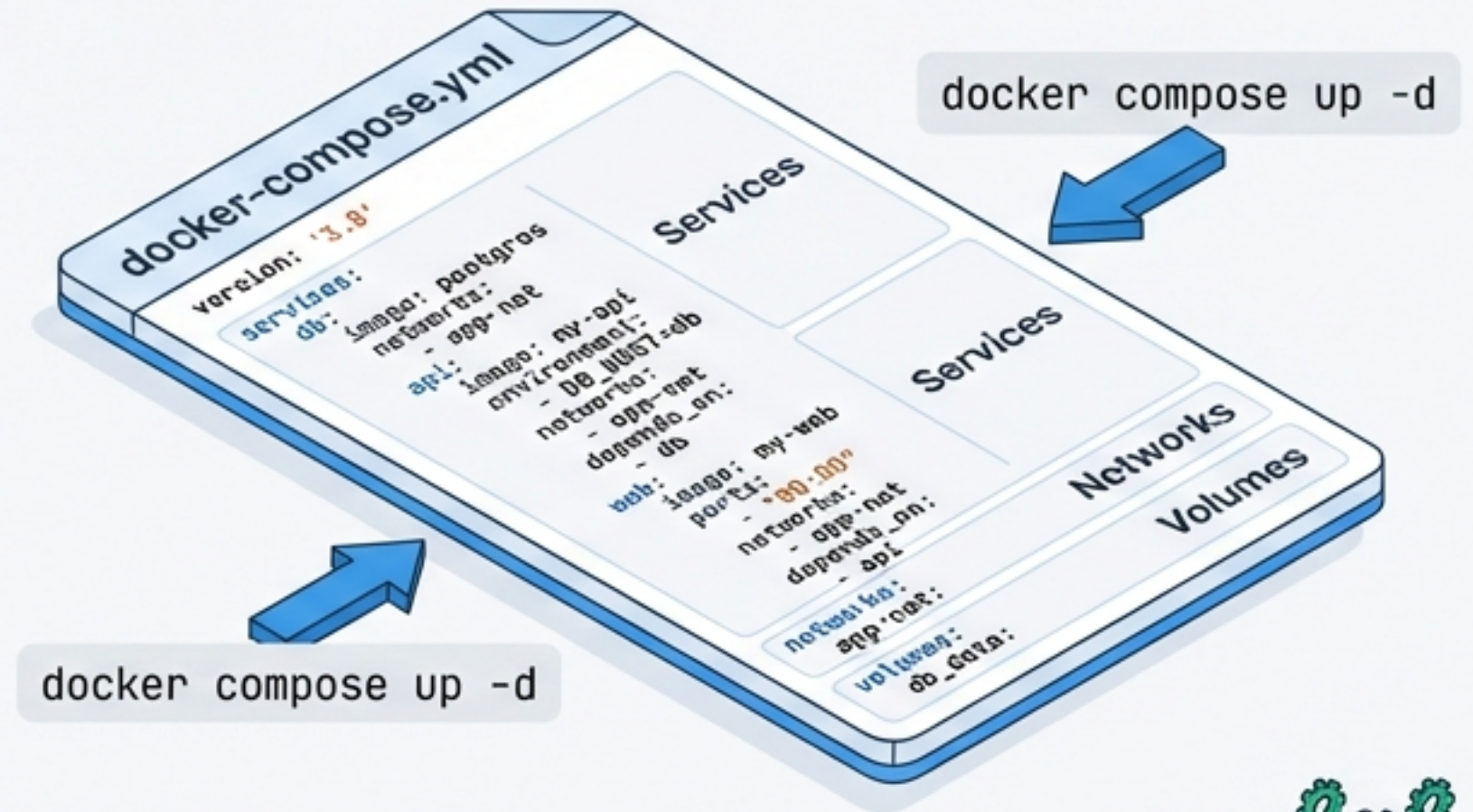
Avant

```
docker run -d --network app-net --name db postgres
docker run -d --network app-net --name db postgres
docker run -d --network app-net -e DB_HOST=db my-api
docker run -d --network app-net -e DB_HOST=db my-api
docker run -d -p 80:80 --network app-net my-web
docker run
```

Scripting manuel & erreurs.



Après



docker compose up -d

Infrastructure as Code. Simple. Reproducible.



Commandes Clés

`docker compose up -d` : Lancer

`docker compose down` : Tout nettoyer

Hygiène et Sécurité en Production

Sécurité (Security Hardening)



Non-Root : `USER node` (Ne jamais tourner en root).



Images Minimales : Utiliser Alpine ou Distroless.



Read-Only : Monter le filesystem en lecture seule quand possible.



Secrets : Pas de mots de passe en variables d'env ! Utiliser Docker Secrets.

Maintenance (Le Ménage)



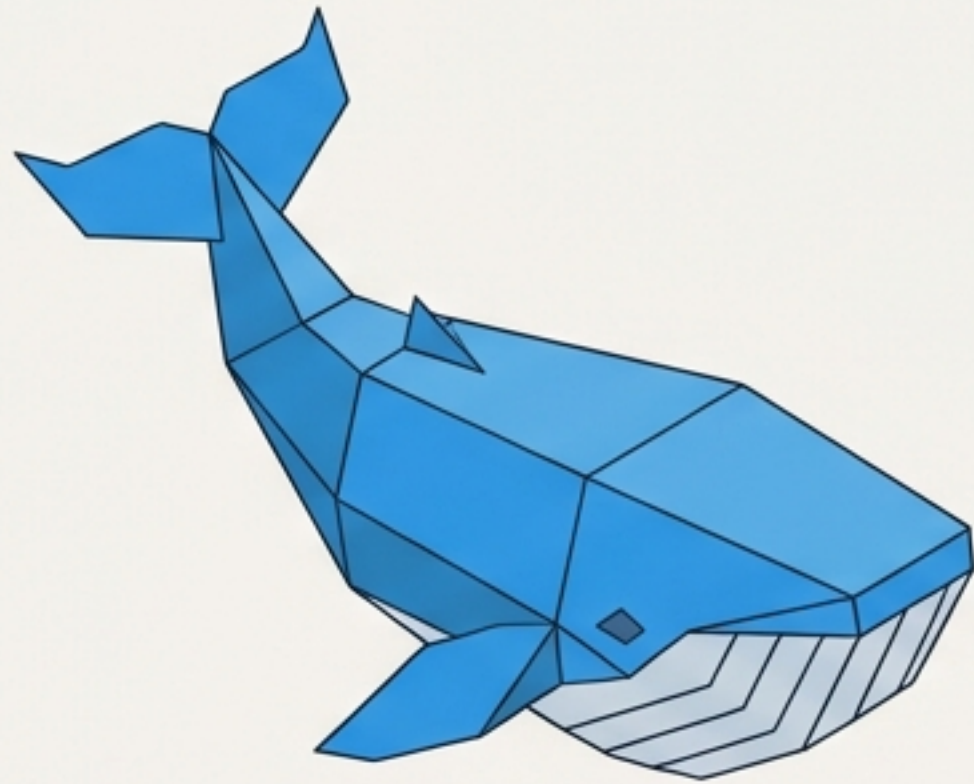
Commande :

```
docker system prune
```

Nettoyeur automatique.
Supprime les conteneurs
arrêtés, réseaux inutiles et
images orphelines.

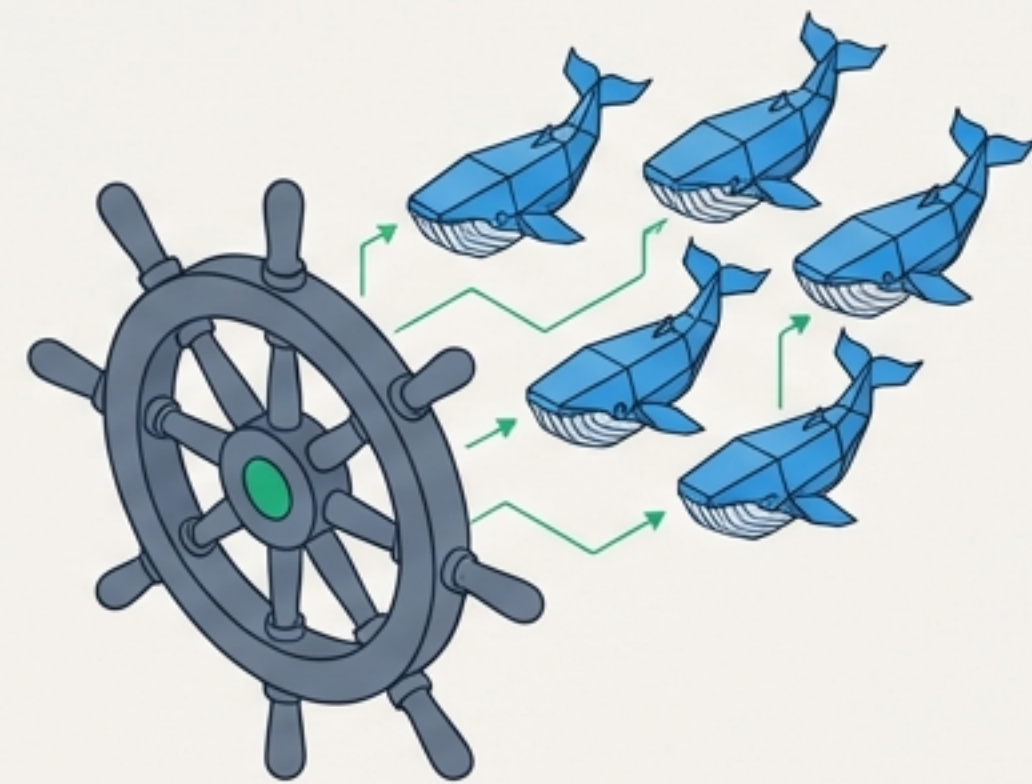
Toujours scanner les vulnérabilités de vos images avant le déploiement.

Au-delà du Conteneur Unique



Docker

Le Standard. Conteneurise l'application. Idéal pour le Dev, le Test, et les apps simples.



Kubernetes

L'Orchestrateur. Gère la flotte. Idéal pour la Haute Dispo, l'Auto-scaling, et le Self-healing.

Maîtriser Docker est le prérequis absolu pour naviguer dans l'ère Cloud Native.